



Md. Manam Shafil Khan

UID	1072410005101014
DEPT	CSE
INSTITUTE	Bangladesh Army University of Science and Technology Khulna
CONTACT	01303677397

Competitive Programming Handbook

Index

Page numbers are shown at the bottom of every page (cover = page 1, index = page 2). Click an entry to jump.

Sections

1. Fast template . . . 3
2. Functions . . . 4
3. Arrays . . . 4
4. Strings . . . 5
5. Vector . . . 5
6. Pair and tuple . . . 6
7. Stack, queue, deque . . . 6
8. Set and multiset . . . 6
9. Map and unordered_map . . . 6
10. Binary search . . . 7
11. GCD / LCM / powers . . . 7
12. Modular arithmetic . . . 7
13. Bit manipulation . . . 7
14. DFS and BFS templates . . . 7
15. Type castings . . . 8
16. Comparators, sorting, common snippets . . . 8
17. Monotonic stack, graph, range, string algos . . . 9
18. Useful builtins . . . 13
19. Useful - data structure choice . . . 13
20. More quick facts (problem -> approach) . . . 14
21. Input and output practice . . . 15
22. Binary <-> decimal conversion (n, i digits) . . . 15
23. Number theory extras . . . 16
24. Graph algorithms extras . . . 17
25. Dynamic programming classics . . . 18
26. Advanced data structures . . . 19
27. String algorithms extras . . . 20
28. Geometry and miscellaneous math . . . 20
29. STL, C++ tricks and debugging . . . 21
30. Handy tables and formulas . . . 22
31. Contest-day checklist . . . 22
32. Search, greedy and counting patterns . . . 23
33. Backtracking, game theory and counting tricks . . . 24
34. More STL algorithms and small helpers . . . 25

Quick find (topic -> page)

Fast I/O + main() . . . 4
Sort + comparator . . . 8
lower_bound / upper_bound . . . 4
Coordinate compression . . . 8
Prefix sum 1D . . . 8
Prefix sum 2D . . . 12
Difference array . . . 11
Kadane . . . 8
Two pointers . . . 9
Sliding window . . . 12
Window maximum . . . 12
Monotonic stack . . . 9
Binary search on answer . . . 12
GCD / LCM . . . 7
Modular power . . . 7
nCr mod p . . . 16
Sieve (basic) . . . 12
Smallest prime factor . . . 16
Euler phi . . . 16
Extended GCD / inverse . . . 16
Miller-Rabin . . . 17
Matrix exponent. . . . 17
Binary -> decimal . . . 16
Decimal -> binary (i digits) . . . 15
bitset as binary number . . . 16
Any base conversion . . . 16
__int128 . . . 12
BFS on grid . . . 10
Dijkstra . . . 10
0-1 BFS . . . 17
Bellman-Ford . . . 11
Floyd Warshall . . . 11
Topological sort . . . 10
DSU . . . 12
Kruskal / Prim (MST) . . . 17
LCA binary lifting . . . 18
SCC (Kosaraju) . . . 18
Bridges . . . 18
Bipartite check . . . 17
Sparse table . . . 10
Fenwick tree (BIT) . . . 11
Segment tree . . . 11
Lazy segment tree . . . 19
Binary trie . . . 19
Ordered set (pbds) . . . 19
KMP . . . 11
Z-function . . . 14
Manacher . . . 20
String hashing . . . 12
Knapsack . . . 18
Coin change . . . 18
LIS . . . 18
LCS / edit distance . . . 18
Bitmask DP (TSP) . . . 19
Digit DP . . . 19
Convex hull . . . 21
Anti-hack hash . . . 7
Stress test / debug . . . 22
Tables: limits, primes . . . 22
Formulas . . . 22
Wrong verdict checks . . . 14
Contest checklist . . . 23

1. Fast template:

```
#pragma GCC optimize("O3,Ofast,unroll-loops")
// [Remove on: compile error / "bad option" warning; WA on decimals -> drop Ofast]
#pragma GCC optimize("fast-math,finite-math-only,no-signed-zeros")
// [Remove on: WA at decimal point / precision error, NaN, inf or -0 printed wrongly]
#pragma GCC optimize("no-trapping-math,associative-math")
// [Remove on: WA in double/long double sums or products (reordering changes rounding)]
#pragma GCC optimize("reciprocal-math,unsafe-math-optimizations")
// [Remove on: decimal point error, a/b not exactly the expected value, WA on sqrt/pow]
#pragma GCC optimize("inline,inline-functions,inline-small-functions")
// [Remove on: compile error / "bad option" warning, or TLE/MLE from code bloat]
#pragma GCC optimize("inline-functions-called-once,partial-inlining")
// [Remove on: compile error / "bad option" warning, or code gets slower]
#pragma GCC optimize("indirect-inlining,early-inlining")
// [Remove on: compile error / "bad option" warning, or code gets slower]
#pragma GCC optimize("tree-vectorize,tree-loop-vectorize,tree-slp-vectorize")
// [Remove on: WA/RE only with this line (vectorizer bug), or compile error]
#pragma GCC optimize("tree-loop-distribution,tree-loop-im")
// [Remove on: compile error / "bad option" warning, or sudden TLE]
#pragma GCC optimize("tree-loop-ivcanon,ivopts")
// [Remove on: compile error / "bad option" warning, or sudden TLE]
#pragma GCC optimize("peel-loops,unswitch-loops,prefetch-loop-arrays")
// [Remove on: compile error, or TLE/MLE (bigger code, cache misses)]
#pragma GCC optimize("move-loop-invariants,predictive-commoning,split-loops")
// [Remove on: compile error / "bad option" warning, or WA only with this line]
#pragma GCC optimize("gcse,gcse-lm,gcse-sm,gcse-las,cse-follow-jumps")
// [Remove on: compile error / "bad option" warning; they are already on in -O2]
#pragma GCC optimize("expensive-optimizations,schedule-insns,schedule-insns2")
// [Remove on: compile error, or slower code / long compile time]
#pragma GCC optimize("rename-registers,reorder-blocks,reorder-functions")
// [Remove on: compile error / "bad option" warning]
#pragma GCC optimize("crossjumping,thread-jumps,strict-aliasing")
// [Remove on: WA/RE after pointer casts or union tricks, or compile error]
#pragma GCC optimize("optimize-sibling-calls,tree-pre,tree-vrp,tree-partial-pre")
// [Remove on: compile error, or WA only with this line (rare optimizer bug)]
#pragma GCC optimize("hoist-adjacent-loads,store-merging")
// [Remove on: compile error / "bad option" warning]
#pragma GCC optimize("devirtualize,devirtualize-speculatively")
// [Remove on: compile error / "bad option" warning (only for virtual functions)]
#pragma GCC optimize("no-stack-protector,omit-frame-pointer")
// [Remove on: "bad option ... to pragma optimize" warning/error]
#pragma GCC optimize("no-plt,no-semantic-interposition")
// [Remove on: "bad option ... to pragma optimize" warning/error]
#pragma GCC optimize("align-functions,align-loops,align-jumps,align-labels")
// [Remove on: compile error / "bad option" warning, or bigger code -> TLE/MLE]
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt,fma,sse4.2,movbe,f16c")
// [Remove on: compile error ("target specific option mismatch" / "inlining failed"),]
// [Runtime Error / "Illegal instruction" (CPU without AVX2), or a non-x86 (ARM) judge]

#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
using ld = long double;
using str = string;
using vll = vector <ll>;
using vc = vector <char>;
using pll = pair <ll, ll>;
using sll = set <ll>;
using usll = unordered_set <ll>;
using mll = multiset <ll>;
using mpll = map <ll, ll>;
using umpll = unordered_map <ll, ll>;
using umpstr = unordered_map <ll, str>;
```

```
#define fi first
#define se second
#define pb push_back
#define pf push_front
#define in insert
// [Remove "in" on: error "expected unqualified-id" / any variable or word named in]
#define sz(x) x.size()
#define all(x) x.begin(),x.end()
// [Remove on: error/WA with sz(a+b) or all(a+b); safe: ((int)(x).size()), (x).begin()]
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    ll t, x, n, i, j, k, total, sum, even, odd;
    ld d, y;
    str s;
    char c;
    bool found;
    cin>>t;
    while (t--) {
        // solve here
    }
    return 0; }
```

Constants used by the snippets below (put above main):

```
const int INF = 1e9, N = 100005, LOG = 20, MOD = 1000000007;
const ll LINF = 4e18;
```

2. Functions:

Pass by value: `void f(int x) { }`
Pass by reference: `void f(int &x) { }`
Pass by const reference: `void f(const vector<int> &v) { }`
Default argument: `void f(int x, int y = 5) { }`

Recursive function:

```
int fact(int n) {
  if (n == 0) return 1;
  return n * fact(n - 1); }
```

3. Arrays:

Max element: `int mx = *max_element(a, a + n);`
Max position: `int pos = max_element(a, a + n) - a;`
Min element: `int mn = *min_element(a, a + n);`
Min position: `int pos = min_element(a, a + n) - a;`
Sort array: `sort(a, a + n);`
Reverse array: `reverse(a, a + n);`
Fill array: `fill(a, a + n, 0);`
Sum of array: `int sum = accumulate(a, a + n, 0);`
Sum (long long): `ll sumll = accumulate(a, a + n, 0LL);`

Bounds (array must be sorted):
Lower bound: `int idx = lower_bound(a, a + n, x) - a;`
Upper bound: `int idx = upper_bound(a, a + n, x) - a;`

Count occurrence in sorted array:

```
int cnt = upper_bound(a, a + n, x) - lower_bound(a, a + n, x);
```

4. Strings:

Input full line: `getline(cin, s); // after cin >> n use cin.ignore(); first`
Length: `int len = s.size();`
Reverse string: `reverse(s.begin(), s.end());`
Sort string: `sort(s.begin(), s.end());`
Substring: `string t = s.substr(pos, len);`
Find substring: `if (s.find("abc") != string::npos) { /* found */ }`
Count characters: `int cnt = count(s.begin(), s.end(), 'a');`
Convert case: `char ch = tolower(c); char ch2 = toupper(c);`
Char checks: `isdigit(c), isalpha(c), islower(c), isupper(c)`
Number to string: `string t = to_string(789);`
String to number: `int x = stoi("123"); ll y = stoll("123456");`

5. Vector:

Create vector: `vector<int> v;`
5 zeros: `vector<int> v2(5);`
5 tens: `vector<int> v3(5, 10);`
Add element: `v.push_back(7); v.emplace_back(8);`
Remove last: `v.pop_back();`
Size: `int n = v.size();`
Access: `cout << v.at(0);`

Sort: `sort(v.begin(), v.end());`
Sort descending: `sort(v.begin(), v.end(), greater<int>());`
Stable sort: `stable_sort(v.begin(), v.end());`
Partial sort: `partial_sort(v.begin(), v.begin() + k, v.end());`
nth_element: `nth_element(v.begin(), v.begin() + k, v.end()); cout << v[k];`
Reverse: `reverse(v.begin(), v.end());`

Unique (remove duplicates after sorting):
```
sort(v.begin(), v.end());
v.erase(unique(v.begin(), v.end()), v.end());
```

Max element: `int mx = *max_element(v.begin(), v.end());`
Max element index: `int idx = max_element(v.begin(), v.end()) - v.begin();`
Min element: `int mn = *min_element(v.begin(), v.end());`
Min element index: `int idx = min_element(v.begin(), v.end()) - v.begin();`

Erase one element: `v.erase(v.begin() + idx);`
Erase range: `v.erase(v.begin() + l, v.begin() + r);`
Count: `int c = count(v.begin(), v.end(), 7);`

Find element:
```
auto it = find(v.begin(), v.end(), x);
if (it != v.end()) { int idx = it - v.begin(); }
```

Lower bound: `auto it = lower_bound(v.begin(), v.end(), x);`
Upper bound: `auto it = upper_bound(v.begin(), v.end(), x);`
Next permutation: `next_permutation(v.begin(), v.end());`
Previous permutation: `prev_permutation(v.begin(), v.end());`

Sum: `int sum = accumulate(v.begin(), v.end(), 0);`
Sum (long long): `ll sumll = accumulate(v.begin(), v.end(), 0LL);`
Partial sum: `vector<int> pref(n); partial_sum(v.begin(), v.end(), pref.begin());`
Fill: `fill(v.begin(), v.end(), -1);`
Iota: `vector<int> id(n); iota(id.begin(), id.end(), 0);`
Left rotate: `rotate(v.begin(), v.begin() + k, v.end());`
Right rotate: `rotate(v.rbegin(), v.rbegin() + k, v.rend());`
Set -> vector: `vector<int> v(s.begin(), s.end());`

Matrix (2D vector):
2D vector: `vector<vector<int>> mat;`
n x m matrix: `vector<vector<int>> mat(n, vector<int>(m));`
n x m with value: `vector<vector<int>> mat(n, vector<int>(m, value));`

Matrix input (0-indexed):
```
vector<vector<int>> a(n, vector<int>(m));
for (int i = 0; i < n; i++) {
  for (int j = 0; j < m; j++) { cin >> a[i][j]; }}
```

Matrix input (1-indexed, int a[i][j] declared with size n+1, m+1):
```
for (int i = 1; i <= n; i++) {
  for (int j = 1; j <= m; j++) { cin >> a[i][j]; }}
```
Read 1-indexed input into 0-indexed matrix: `cin >> a[i - 1][j - 1];`

Rows and columns of an already given matrix:
Number of rows: `int rows = mat.size();`
Number of columns: `int cols = mat[0].size(); // check rows > 0 first`
Vector of strings (grid): `rows = g.size(); cols = g[0].size();`
C array (same scope only): `int arr[3][4];`
`int rows = sizeof(arr) / sizeof(arr[0]); // 3`
`int cols = sizeof(arr[0]) / sizeof(arr[0][0]); // 4`
Empty-safe: `int cols = rows ? mat[0].size() : 0;`

6. Pair and tuple:

Pair: `pair<int,int> p = {3, 5}; cout << p.first << ' ' << p.second;`
Make pair: `auto p = make_pair(3, 5);`
Unpack (C++17): `auto [a, b] = p;`
Vector of pairs: `vector<pair<int,int>> vp; vp.push_back({1, 2});`
Sort pairs: `sort(vp.begin(), vp.end());`

Custom pair sort (first ascending, second descending):
```
sort(vp.begin(), vp.end(), [](const pair<int,int> &a, const pair<int,int> &b) {
  if (a.first != b.first) return a.first < b.first;
  return a.second > b.second; });
```

Tuple: `tuple<int,int,int> t = {1, 2, 3}; cout << get<0>(t);`

7. Stack, queue, deque:

Stack: `stack<int> st; st.push(10); st.pop(); cout << st.top();`
Queue: `queue<int> q; q.push(10); q.pop(); cout << q.front();`
Deque: `deque<int> dq; dq.push_front(1); dq.push_back(2); dq.pop_front(); dq.pop_back();`

Priority queue:
Max-heap: `priority_queue<int> pq; pq.push(5); pq.push(1); cout << pq.top(); // largest`
Min-heap: `priority_queue<int, vector<int>, greater<int>> pq;`
Pair in priority queue: `priority_queue<pair<int,int>> pq;`
Pair min-heap:
`priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> mpq;`
With template alias: `priority_queue<pll, vector<pll>, greater<pll>> pq;`

8. Set and multiset:

Set: `set<int> s; s.insert(5); s.insert(1); s.erase(5);`
Check existence: `if (s.count(5)) { /* exists */ }`
Lower bound (first >= 4): `auto it = s.lower_bound(4);`
Upper bound (first > 4): `auto it2 = s.upper_bound(4);`
Smallest / largest: `*s.begin(), *s.rbegin()`
Use s.lower_bound(x), NOT lower_bound(s.begin(), s.end(), x) (that one is O(n)).

Multiset: `multiset<int> ms; ms.insert(5); ms.insert(5);`
Erase ONE copy: `ms.erase(ms.find(5)); // ms.erase(5) removes ALL copies`

9. Map and unordered_map:

Map: `map<int, string> mp; mp[1] = "one"; mp[2] = "two";`
Access / update: `cout << mp[1]; // mp[k] creates k if it is missing`
Check if key exists: `if (mp.count(3)) { /* key exists */ }`
Unordered map: `unordered_map<int, int> ump; // can be anti-hashed, see below`

Iterate map:
```
for (auto &x : mp) { cout << x.first << ' ' << x.second << '\n'; }
```

Anti-hack hash for unordered_map (unordered_map<ll, int, chash> m):

```
struct chash {
  static ull sp(ull x) { x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9; x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31); }
  size_t operator()(ull x) const {
    static const ull R = chrono::steady_clock::now().time_since_epoch().count();
    return sp(x + R); }};
```

10. Binary search:

Binary search (sorted container): `binary_search(v.begin(), v.end(), x); // true / false`
First >= x: `auto lb = lower_bound(v.begin(), v.end(), x);`
First > x: `auto ub = upper_bound(v.begin(), v.end(), x);`

11. GCD / LCM / powers:

GCD: `int g = gcd(a, b);`
LCM: `long long l = lcm(a, b); // safer: a / gcd(a, b) * b`

Fast exponentiation (no mod, watch overflow):

```
ll binpow(ll a, ll b) {
  ll res = 1;
  while (b > 0) { if (b & 1) res = res * a;
    a = a * a; b >>= 1; }
  return res; }
```

Modular power:

```
ll modpow(ll a, ll b, ll mod) {
  ll res = 1 % mod; a %= mod;
  while (b > 0) { if (b & 1) res = res * a % mod;
    a = a * a % mod; b >>= 1; }
  return res; }
```

12. Modular arithmetic:

Common modulus: `const int MOD = 1000000007; // also 998244353`
Add safely: `x = (x + y) % MOD;`
Subtract safely: `x = (x - y + MOD) % MOD;`
Multiply safely: `x = (1LL * x * y) % MOD;`
Modular inverse (MOD prime): `ll inv = modpow(a, MOD - 2, MOD);`
Modular division: `x = x * modpow(y, MOD - 2, MOD) % MOD;`

13. Bit manipulation:

Check bit: `if (x & (1 << k)) { /* k-th bit is set */ }`
Set bit: `x |= (1 << k);`
Clear bit: `x &= ~(1 << k);`
Toggle bit: `x ^= (1 << k);`
Lowest set bit: `int low = x & -x;`
Count set bits: `int cnt = __builtin_popcount(x);`
Count set bits (ll): `int cntll = __builtin_popcountll(x);`
For long long use 1LL << k (1 << k overflows when k >= 31).

14. DFS and BFS templates:

Add edge (undirected): `adj[u].push_back(v); adj[v].push_back(u);`

DFS on graph:

```
vector<int> adj[100005];
vector<int> vis(100005, 0);
void dfs(int u) {
  vis[u] = 1;
  for (int v : adj[u]) { if (!vis[v]) dfs(v); }}
```

BFS on graph:
```
queue<int> q; vector<int> dist(100005, -1);
dist[start] = 0; q.push(start);
while (!q.empty()) {
  int u = q.front(); q.pop();
  for (int v : adj[u]) {
    if (dist[v] == -1) { dist[v] = dist[u] + 1; q.push(v); }}}
```

15. Type castings:

int -> double: `(double)x`
double -> int: `(int)x // cuts the decimal part`
int -> long long: `(long long)x`
char digit -> int: `c - '0'`
int digit -> char: `x + '0'`
Modern cast: `static_cast<double>(x)`
a * b with ints overflows BEFORE it is stored in ll -> write 1LL * a * b.

16. Comparators, sorting and common snippets:

Comparator functions:
```
bool cmp(pair<int,int> a, pair<int,int> b) { return a.second < b.second; } // asc by
    second
bool cmp(int a, int b) { return a > b; } // descending
```
Usage: `sort(v.begin(), v.end(), cmp);`

Lambda sort:
```
sort(v.begin(), v.end(), [](pair<int,int> a, pair<int,int> b) {
  return a.second < b.second; });
```

Custom comparator pair (first ascending, second descending):
```
bool cmp(pair<int,int> a, pair<int,int> b) {
  if (a.first == b.first) return a.second > b.second;
  return a.first < b.first; }
```

Coordinate compression (when numbers are huge):
```
vector<int> temp = v; sort(temp.begin(), temp.end());
temp.erase(unique(temp.begin(), temp.end()), temp.end());
for (auto &x : v) { x = lower_bound(temp.begin(), temp.end(), x) - temp.begin(); }
```

Frequency of elements (map):
```
map<int,int> freq; for (int x : v) { freq[x]++; }
```

Most frequent element:
```
int mx = 0, ans = -1;
for (auto x : freq) { if (x.second > mx) { mx = x.second; ans = x.first; }}
```

Character frequency:
```
int freq[26] = {0}; for (char c : s) { freq[c - 'a']++; }
```
Frequency array: `int freq[100005] = {0}; freq[x]++;`

Prefix maximum:
```
pref[0] = a[0]; for (int i = 1; i < n; i++) { pref[i] = max(pref[i-1], a[i]); }
```

Suffix maximum:
```
suf[n-1] = a[n-1]; for (int i = n - 2; i >= 0; i--) { suf[i] = max(suf[i+1], a[i]); }
```

1D prefix sum:
```
vector<int> pref(n + 1, 0);
for (int i = 0; i < n; i++) { pref[i + 1] = pref[i] + a[i]; }
```
Range sum [l, r]: `int sum = pref[r + 1] - pref[l]; // 1-indexed: prefix[r] - prefix[l-1]`

Prefix XOR:
```
px[0] = 0; for (int i = 1; i <= n; i++) { px[i] = px[i-1] ^ a[i]; }
```
Range XOR: `px[r] ^ px[l-1]`

Kadane's algorithm (maximum subarray sum):
```
ll best = -1e18, cur = 0;
for (int x : v) { cur = max((ll)x, cur + x); best = max(best, cur); }
```

Generate all subarrays:
```
for (int i = 0; i < n; i++) { for (int j = i; j < n; j++) { /* subarray i to j */ }}
```

Generate all subsequences (bitmask method):
```
for (int mask = 0; mask < (1 << n); mask++) {
  for (int i = 0; i < n; i++) { if (mask & (1 << i)) cout << a[i]; }}
```

Decimal to binary: `bitset<32> b(n); cout << b;`
Binary to decimal: `int x = stoi(s, 0, 2);`
Check perfect square: `int x = sqrt(n); if (x * x == n) { /* perfect square */ }`

nCr (combination):
```
ll nCr(int n, int r) {
  ll res = 1;
  for (int i = 0; i < r; i++) { res *= n - i; res /= i + 1; }
  return res; }
```

Count digits: `int digits = log10(n) + 1;`

Sum of digits:
```
int sum = 0; while (n) { sum += n % 10; n /= 10; }
```

Reverse number:
```
int rev = 0; while (n) { rev = rev * 10 + n % 10; n /= 10; }
```

Palindrome number:
```
int temp = n, rev = 0;
while (temp) { rev = rev * 10 + temp % 10; temp /= 10; }
if (rev == n) { /* palindrome */ }
```

Merge intervals:
```
sort(v.begin(), v.end()); vector<pair<int,int>> ans;
for (auto p : v) {
  if (ans.empty() || ans.back().second < p.first) ans.push_back(p);
  else ans.back().second = max(ans.back().second, p.second); }
```

17. Monotonic stack, graph, range and string algorithms:

Monotonic stack - nearest greater element:
```
stack<int> st;
for (int i = 0; i < n; i++) {
  while (!st.empty() && st.top() < a[i]) st.pop();
  // st.top() here = previous greater element of a[i] (if st is not empty)
  st.push(a[i]); }
```

Cycle detection (undirected graph):
```
bool dfs(int u, int p) {
  vis[u] = 1;
  for (int v : adj[u]) {
    if (!vis[v]) { if (dfs(v, u)) return true; }
    else if (v != p) return true; }
  return false; }
```

Connected components:
```
int cnt = 0;
for (int i = 1; i <= n; i++) { if (!vis[i]) { dfs(i); cnt++; }}
```

Two pointers (sorted array, pair with sum = target):
```
int l = 0, r = n - 1;
while (l < r) {
  if (a[l] + a[r] == target) break;
  else if (a[l] + a[r] < target) l++;
  else r--; }
```

Tree basics: `Nodes: n, Edges: n-1, Leaf: degree == 1`

Check prime:
```
bool prime(int n) {
  if (n < 2) return false;
  for (int i = 2; i * i <= n; i++) { if (n % i == 0) return false; }
  return true; }
```

Prime factorization (print):
```
for (int i = 2; i * i <= n; i++) { while (n % i == 0) { cout << i << " "; n /= i; }}
if (n > 1) cout << n;
```

Prime factors into a vector:
```
vector<int> factors;
for (int i = 2; i * i <= n; i++) { while (n % i == 0) { factors.push_back(i); n /= i;
    }}
if (n > 1) factors.push_back(n);
```

Divisors of a number (count = size):
```
vector<int> divs;
for (int i = 1; i * i <= n; i++) {
  if (n % i == 0) { divs.push_back(i); if (i != n / i) divs.push_back(n / i); }}
```

Number of divisors only:
```
int divisors = 0;
for (int i = 1; i * i <= n; i++) { if (n % i == 0) { divisors++; if (i != n / i)
    divisors++; }}
```

Direction arrays (grid problems):
Up Down Left Right: `int dx[] = {-1, 1, 0, 0}, dy[] = {0, 0, -1, 1};`
8 directions: `int dx[] = {-1,-1,-1,0,0,1,1,1}, dy[] = {-1,0,1,-1,1,-1,0,1};`

BFS on grid:
```
queue<pair<int,int>> q; q.push({sx, sy});
while (!q.empty()) {
  auto [x, y] = q.front(); q.pop();
  for (int i = 0; i < 4; i++) {
    int nx = x + dx[i], ny = y + dy[i];
    if (nx < 0 || ny < 0 || nx >= n || ny >= m) continue;
    // check wall / vis, then dist[nx][ny] = dist[x][y] + 1; q.push({nx, ny}); }}}
```

Dijkstra:
```
vector<pair<int,int>> adj[N]; // adj[u].push_back({v, w});
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
vector<int> dist(n + 1, INF); dist[src] = 0; pq.push({0, src});
while (!pq.empty()) {
  auto [d, u] = pq.top(); pq.pop();
  if (d > dist[u]) continue;
  for (auto [v, w] : adj[u]) {
    if (dist[v] > dist[u] + w) { dist[v] = dist[u] + w; pq.push({dist[v], v}); }}}
```

Topological sort (Kahn):
```
queue<int> q;
for (int i = 1; i <= n; i++) { if (indegree[i] == 0) q.push(i); }
while (!q.empty()) {
  int u = q.front(); q.pop(); // u comes next in the order
  for (int v : adj[u]) { indegree[v]--; if (indegree[v] == 0) q.push(v); }}
```

Sparse table (RMQ):
```
for (int i = 0; i < n; i++) st[i][0] = a[i];
for (int j = 1; j < LOG; j++) {
  for (int i = 0; i + (1 << j) <= n; i++) {
    st[i][j] = min(st[i][j-1], st[i + (1 << (j-1))][j-1]); }}
```
Query: `int k = log2(r - l + 1); int ans = min(st[l][k], st[r - (1 << k) + 1][k]);`

Fenwick tree (BIT) - point update + prefix sum:

```
vector<int> bit(n + 1);
void add(int idx, int val) { while (idx <= n) { bit[idx] += val; idx += idx & -idx; }}
int sum(int idx) {
  int res = 0;
  while (idx > 0) { res += bit[idx]; idx -= idx & -idx; }
  return res; }
```

Range sum: `sum(r) - sum(l - 1);`

Segment tree (sum, point update + range query; use node = 1, range [0, n-1]):

```
ll t[4 * N]; // a[] is the input array
void build(int node, int l, int r) {
  if (l == r) { t[node] = a[l]; return; }
  int m = (l + r) / 2;
  build(node*2, l, m); build(node*2+1, m+1, r);
  t[node] = t[node*2] + t[node*2+1]; }
void update(int node, int l, int r, int pos, ll val) {
  if (l == r) { t[node] = val; return; }
  int m = (l + r) / 2;
  if (pos <= m) update(node*2, l, m, pos, val);
  else update(node*2+1, m+1, r, pos, val);
  t[node] = t[node*2] + t[node*2+1]; }
ll query(int node, int l, int r, int ql, int qr) {
  if (qr < l || r < ql) return 0;
  if (ql <= l && r <= qr) return t[node];
  int m = (l + r) / 2;
  return query(node*2, l, m, ql, qr) + query(node*2+1, m+1, r, ql, qr); }
```

Calls: `build(1, 0, n-1); update(1, 0, n-1, pos, val); query(1, 0, n-1, l, r);`
Left = node * 2, Right = node * 2 + 1. For min/max: use min()/max() and return INF / -INF outside the range.

Floyd Warshall (all-pairs shortest path; dist[i][i] = 0, others INF):

```
for (int k = 1; k <= n; k++) { for (int i = 1; i <= n; i++) { for (int j = 1; j <= n;
    j++) {
  dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]); }}}
```

With int keep INF <= 1e9 (INF + INF must not overflow).

KMP string matching - build LPS (longest prefix suffix) array:

```
vector<int> buildLPS(const string &pat) {
  int n = pat.size(); vector<int> lps(n, 0); int len = 0, i = 1;
  while (i < n) {
    if (pat[i] == pat[len]) lps[i++] = ++len;
    else if (len) len = lps[len - 1];
    else lps[i++] = 0; }
  return lps; }
```

KMP search (find all occurrences of pat in text):

```
void KMP(const string &text, const string &pat) {
  auto lps = buildLPS(pat); int i = 0, j = 0, n = text.size(), m = pat.size();
  while (i < n) {
    if (text[i] == pat[j]) { i++; j++; }
    if (j == m) { cout << "Found at index " << i - j << "\n"; j = lps[j - 1]; }
    else if (i < n && text[i] != pat[j]) { if (j) j = lps[j - 1]; else i++; }}}
```

Difference array (many range updates):

```
vector<int> diff(n + 1, 0);
diff[l] += val; diff[r + 1] -= val;
for (int i = 1; i < n; i++) { diff[i] += diff[i-1]; }
```

Bellman-Ford (negative edges; edges = {u, v, w}):

```
vector<int> dist(n + 1, INF); dist[src] = 0;
for (int i = 0; i < n - 1; i++) {
  for (auto [u, v, w] : edges) { if (dist[u] + w < dist[v]) dist[v] = dist[u] + w; }}
```

One more round that still relaxes an edge -> negative cycle exists.

2D prefix sum:
```
for (int i = 1; i <= n; i++) {
  for (int j = 1; j <= m; j++) {
    pref[i][j] = a[i][j] + pref[i-1][j] + pref[i][j-1] - pref[i-1][j-1]; }}
int sum = pref[r2][c2] - pref[r1-1][c2] - pref[r2][c1-1] + pref[r1-1][c1-1];
```

String hashing:
```
const ll HM = 1e9 + 7, BASE = 31;
vector<ll> h(n + 1, 0), pw(n + 1, 1);
for (int i = 0; i < n; i++) {
  h[i+1] = (h[i] * BASE + (s[i] - 'a' + 1)) % HM;
  pw[i+1] = pw[i] * BASE % HM; }
auto get = [&](int l, int r) { return (h[r+1] - h[l] * pw[r-l+1] % HM + HM * 2) % HM;
    };
```

Sliding window - fixed size k:
```
ll sum = 0; for (int i = 0; i < k; i++) sum += a[i]; // first window
for (int i = k; i < n; i++) { sum += a[i]; sum -= a[i - k]; /* use sum */ }
```

Sliding window - variable size:
```
int left = 0;
for (int right = 0; right < n; right++) {
  add(a[right]);
  while (condition) { remove(a[left]); left++; }}
```

Window maximum:
```
deque<int> dq;
for (int i = 0; i < n; i++) {
  while (!dq.empty() && dq.front() < i - k + 1) dq.pop_front();
  while (!dq.empty() && a[dq.back()] < a[i]) dq.pop_back();
  dq.push_back(i);
  if (i >= k - 1) cout << a[dq.front()] << " "; }
```

__int128 for huge numbers:
```
__int128 x = (__int128)1e36;
void print128(__int128 x) {
  if (x < 0) { cout << '-'; x = -x; }
  if (x > 9) print128(x / 10);
  cout << (char)('0' + x % 10); }
```

Binary search on answer (smallest value where check() is true):
```
int lo = 0, hi = 1e9;
while (lo < hi) {
  int mid = lo + (hi - lo) / 2;
  if (check(mid)) hi = mid; else lo = mid + 1; }
cout << lo;
```

Disjoint set union (DSU):
```
vector<int> parent, siz;
void dsu_init(int n) {
  parent.resize(n + 1); siz.assign(n + 1, 1);
  iota(parent.begin(), parent.end(), 0); }
int find_set(int v) { return v == parent[v] ? v : parent[v] = find_set(parent[v]); }
void union_set(int a, int b) {
  a = find_set(a); b = find_set(b);
  if (a != b) { if (siz[a] < siz[b]) swap(a, b); parent[b] = a; siz[a] += siz[b]; }}
```

Sieve of Eratosthenes:
```
vector<bool> is_prime(n + 1, true); is_prime[0] = is_prime[1] = false;
for (ll i = 2; i * i <= n; i++) {
  if (is_prime[i]) { for (ll j = i * i; j <= n; j += i) is_prime[j] = false; }}
```

18. Useful builtins:

Number of 1 bits: `__builtin_popcount(x)`
Leading zeros: `__builtin_clz(x)`
Trailing zeros: `__builtin_ctz(x)`
Returns odd/even parity: `__builtin_parity(x)`
For long long add ll: __builtin_popcountll, clzll, ctzll. Highest set bit: 31 - __builtin_clz(x) / 63 - __builtin_clzll(x).

19. Useful - data structure choice:

Trie -> dictionary, prefix matching, XOR problems. Operations: insert(word), search(word), startsWith(prefix)
Check same component -> find_set(a) == find_set(b) | Merge -> union_set(a, b)
Unweighted shortest path -> BFS | Weighted shortest path -> Dijkstra | Negative edges -> Bellman Ford
MST -> Kruskal / Prim | DAG -> Topological Sort | Components -> DFS / DSU
Range sum -> Prefix Sum | Range update -> Difference Array | Max subarray -> Kadane
Sorted array -> Binary Search | Frequency -> Map

Array / vector problems (keyword -> approach):
"Subarray", "continuous", "contiguous" -> Sliding Window / Prefix Sum
"Maximum sum subarray" -> Kadane's Algorithm
"Range sum query" -> Prefix Sum | "Many range updates" -> Difference Array
"Find pair with sum X" -> Two Pointer / Hash Map | "Find triplets" -> Sorting + Two Pointer
"Longest/shortest subarray satisfying condition" -> Sliding Window
"Sorted array + search" -> Binary Search
"Find minimum / maximum possible answer" -> Binary Search on Answer
"Remove duplicates" -> Set / sort + unique | "Frequency of elements", "most frequent" -> Map
"K-th smallest/largest" -> nth_element / Priority Queue | "Top K elements" -> Heap

Sorting patterns:
Need elements in order -> sort() | Need custom ordering -> Comparator / Lambda
Intervals overlap -> Merge Intervals | Schedule maximum activities -> Greedy + Sorting
Closest pair / median problems -> Sorting + Two Pointer

Sliding window recognition:
Longest substring | Shortest substring | Maximum sum of k elements | At most k distinct elements
No repeating characters | Continuous segment

Binary search recognition:
Minimum possible | Maximum possible | Smallest value satisfying condition
Largest value satisfying condition | Can we do it in X days? | Maximum distance possible

Tree problems:
Tree traversal -> DFS | Level order traversal -> BFS | Height/depth -> DFS
Diameter of tree -> DFS (twice) | Lowest Common Ancestor -> Binary Lifting
Subtree queries -> DFS + Euler Tour | Tree distances -> DFS/BFS

Graph problems:
Connected components -> DFS/BFS/DSU | Shortest path (no weight) -> BFS
Shortest path (positive weight) -> Dijkstra | Negative edge -> Bellman-Ford
All pair shortest path -> Floyd Warshall | Detect cycle undirected -> DFS/DSU
Detect cycle directed -> DFS colors | Dependency ordering -> Topological Sort
Minimum spanning tree -> Kruskal/Prim | Dynamic connectivity -> DSU

Grid problems (matrix, maze, island, cells, movement, neighbors):
Shortest path -> BFS | Flood fill -> DFS/BFS | Connected regions -> DFS/BFS
Use int dx[] = {-1,1,0,0}; int dy[] = {0,0,-1,1};

Greedy recognition:
Maximum number | Minimum operations | Choose optimal | Best possible | Arrange | Interval scheduling
Tools -> Sorting, Priority Queue, Greedy choice

Stack problems:
Nearest | Previous | Next | Greater | Smaller | Remove duplicates | Balanced brackets
Tools -> Stack, Monotonic Stack

Math recognition:
Prime numbers -> Sieve | Prime factors -> Factorization | Divisors -> sqrt(n) loop
GCD -> gcd() | Huge power -> Binary exponentiation | Modular arithmetic -> modpow
Combination -> nCr | Number bits -> Bit manipulation

Bit manipulation:
Keywords: binary, bits, power of two, subset, xor, x&(x-1)
Count bits -> popcount | Check power of 2 -> x&(x-1)==0 | Generate subsets -> Bitmask
XOR problems -> Bit manipulation

Dynamic programming recognition:
Maximum/minimum ways | Number of ways | Choose or don't choose | Optimal answer | Previous states
Fibonacci -> Linear DP | Knapsack -> 0/1 DP | Coin change -> Unbounded DP
LIS -> Longest Increasing Subsequence | Grid paths -> Grid DP | String comparison -> LCS/Edit Distance

Advanced data structures:
Range minimum query -> Sparse Table | Range update/query -> Segment Tree
Frequency updates -> Fenwick Tree | Prefix matching -> Trie | Huge connectivity -> DSU

Contest trigger words:
"minimum operations" -> Greedy / DP | "maximum value" -> Greedy / DP / Binary Search
"number of ways" -> DP / Combinatorics | "all possible" -> Backtracking / Bitmask
"subsequence" -> DP | "substring" -> Sliding Window / KMP | "sorted" -> Binary Search / Two Pointer
"graph" -> DFS/BFS | "shortest" -> BFS/Dijkstra | "connect" -> DSU
"range" -> Prefix Sum / Segment Tree | "k-th" -> Binary Search / Heap / nth_element

Final contest thinking process:
1. Is it sorted? -> Binary Search | 2. Is it continuous? -> Sliding Window
3. Is it asking range? -> Prefix Sum / Segment Tree | 4. Is it graph? -> BFS / DFS / Dijkstra
5. Is it choosing optimal? -> Greedy / DP | 6. Need frequency? -> Map | 7. Need connectivity? -> DSU

20. More quick facts (problem type -> approach):

Constraints -> expected complexity (about 1e8 simple operations per second):
n <= 10 -> O(n!) brute force | n <= 20 -> O(2^n) bitmask / backtracking
n <= 500 -> O(n^3) (Floyd Warshall) | n <= 5000 -> O(n^2) | n <= 1e5 -> O(n log n)
n <= 1e6 -> O(n) or O(n log n) with small constant | n <= 1e9 -> O(sqrt n) or O(log n)

More "kind of problem -> approach":
Count subarrays with sum = k -> prefix sum + map | Subarray sum divisible by k -> prefix % k counts
Equal number of 0s and 1s -> change 0 to -1, prefix sum + map | Count pairs with sum/difference -> sort + two pointers / map
Previous/next greater or smaller -> monotonic stack | Max/min of every window of size k -> deque
Maximize the minimum / minimize the maximum -> binary search on answer
Intervals overlap count / busy time -> sweep line / difference array
Number of inversions -> merge sort / Fenwick tree | Minimum operations to make all equal -> median (sort)
Bipartite / 2-colorable -> BFS coloring | Weights only 0 and 1 -> 0-1 BFS with deque
Connect everything at minimum cost -> MST (Kruskal + DSU) | Tree diameter -> two BFS/DFS
Many queries, small n (<= 400) -> Floyd Warshall | Many range min/gcd queries -> Sparse Table
Lexicographically smallest -> greedy / stack | Brackets matching -> stack
Pattern in string -> KMP / hashing / Z-function | Palindromes -> two pointers / DP
Choose items with a weight limit -> knapsack DP | Probability / expected value -> DP + modular inverse
Digit property of numbers up to 1e18 -> digit DP | Offline queries -> sort queries, then sweep

Useful formulas:
Sum 1..n = n(n+1)/2 | Sum of squares = n(n+1)(2n+1)/6 | Subarrays = n(n+1)/2 | Subsets = 2^n
Pairs = n(n-1)/2 | ceil(a/b) = (a + b - 1) / b | gcd(a,b) * lcm(a,b) = a * b
Primes below 1e6 = 78498 | A number up to 1e9 has at most 1344 divisors
INT_MAX = 2147483647 (about 2.1e9) | LLONG_MAX = 9.22e18 | sum of 1e5 values of 1e9 needs ll

STL complexity:
vector push_back O(1) amortized | set/map/priority_queue O(log n) | unordered_map O(1) average
sort O(n log n) | lower_bound on vector O(log n) | erase in the middle of a vector O(n)

Wrong verdict -> first checks:
Wrong Answer -> overflow (use ll), n = 1 / empty input, 0 vs 1 indexing, uninitialized variables, reset between tests
Runtime Error -> index out of range, division by zero, deep recursion (DFS depth ~1e5+ -> iterative), s.size() - 1 when size is 0
Time Limit -> complexity too high, endl instead of '\n', cin without sync_with_stdio(false), map where array works
Memory Limit -> huge arrays, vector<vector> with many tiny rows, long long where int works
Precision -> cout << fixed << setprecision(9) << x; compare doubles with abs(a - b) < 1e-9
Read until end of input: while (cin >> x) { } | Yes/No: cout << (ok ? "YES" : "NO") << '\n';

21. Input and output practice:

Pair input / print:

```
pair<int,int> p; cin >> p.first >> p.second;
cout << "Pair:\n"; cout << p.first << " " << p.second << "\n\n";
```

Vector input / print:

```
cin >> n; vector<int> v;
for (int i = 0; i < n; i++) { int x; cin >> x; v.push_back(x); }
for (auto x : v) { cout << x << " "; }
```

Vector of pairs input / print:

```
cin >> n; vector<pair<int,int>> vp;
for (int i = 0; i < n; i++) { int x, y; cin >> x >> y; vp.push_back({x, y}); }
for (auto p : vp) { cout << p.first << " " << p.second << '\n'; }
```

Set input / print (multiset<int> ms works the same way):

```
cin >> n; set<int> s;
for (int i = 0; i < n; i++) { int x; cin >> x; s.insert(x); }
for (auto x : s) { cout << x << " "; }
```

Map input / print:

```
cin >> n; map<int, string> mp;
for (int i = 0; i < n; i++) { int key; string value; cin >> key >> value; mp[key] =
    value; }
for (auto p : mp) { cout << p.first << " " << p.second << '\n'; }
```

Matrix input / print:

```
int r, c; cin >> r >> c; vector<vector<int>> mat(r, vector<int>(c));
for (int i = 0; i < r; i++) { for (int j = 0; j < c; j++) { cin >> mat[i][j]; }}
for (auto row : mat) { for (auto x : row) { cout << x << " "; }}
```

Iterator print:

```
pair<int,int> p = {10, 20}; cout << p.first << " " << p.second << "\n\n";
```

Vector: `for (auto it = v.begin(); it != v.end(); it++) { cout << *it << " "; }`

Vector of pairs:

```
for (auto it = vp.begin(); it != vp.end(); it++) {
  cout << it->first << " " << it->second << '\n'; }
```

Set / multiset: `for (auto it = s.begin(); it != s.end(); it++) { cout << *it << " "; }`

Map:

```
for (auto it = mp.begin(); it != mp.end(); it++) {
  cout << it->first << " " << it->second << '\n'; }
```

Matrix:

```
for (auto row = mat.begin(); row != mat.end(); row++) {
  for (auto col = row->begin(); col != row->end(); col++) { cout << *col << " "; }}
```

22. Binary <-> decimal conversion (n and i digits):

Decimal n -> binary string with exactly i digits (leading zeros kept):

```
string toBin(ll n, int i) {
  string r(i, '0');
  for (int b = 0; b < i; b++) if (n >> b & 1) r[i - 1 - b] = '1';
  return r; }
```

n must be >= 0 and < 2^i (i <= 62), otherwise the high bits are cut off.

Read n and i, print the i-digit binary (n = 5, i = 8 -> 00000101):

```
cin >> n >> i; string b = toBin(n, i); cout << b << '\n';
```

Same with bitset (needs a constant size, so cut the left zeros):

```
string b2 = bitset<64>(n).to_string().substr(64 - i); // i <= 64
bitset<8> fx(n); cout << fx; // when i is a fixed constant (here 8)
```

Use it like a normal number, way 1: digits as a decimal-looking number (i <= 18):

```
ll bin = stoll(b); // "00000101" -> 101, a plain ll: + - * / % and comparisons work
cout << setw(i) << setfill('0') << bin << '\n'; // 00000101 (the zeros come back)
ll ones = 0; for (ll t = bin; t; t /= 10) ones += t % 10; // number of 1 bits
```

setfill stays active after use: reset it with cout << setfill(' ');

Way 2: keep the value in n and use bit operators (this IS binary arithmetic):

```
n & m, n | m, n ^ m, n << 1, n >> 1, n & (1LL << k), __builtin_popcountll(n)
for (int k = i - 1; k >= 0; k--) cout << (n >> k & 1); // print i bits, no string
```

Way 3: bitset works like an i-bit binary integer:

```
bitset<8> a(5), c("1100"); // a = 00000101, c = 00001100 (from decimal / from string)
a | c, a & c, a ^ c, a << 2, a >> 1, ~a, a == c // all bit operators work directly
a.count(); a.any(); a.none(); a[k]; a.set(k); a.reset(k); a.flip(k); a.size();
ll dec = a.to_ullong(); string t = a.to_string(); // back to decimal / binary text
```

Binary string -> decimal:

```
ll x = stoll(s, nullptr, 2); // "101101" -> 45 (s non-empty, at most 62 digits)
ll fromBin(const string &s) {
  ll r = 0; for (char c : s) r = r * 2 + (c - '0');
  return r; }
```

Binary written as a number <-> decimal (101101 <-> 45, up to 18 binary digits):

```
ll fromBinNum(ll b) { ll r = 0, p = 1;
  for (; b; b /= 10, p *= 2) r += b % 10 * p;
  return r; }
ll toBinNum(ll n) { ll r = 0, p = 1;
  for (; n; n /= 2, p *= 10) r += n % 2 * p;
  return r; }
```

Any base 2..36 (hex, octal, base 3 ...):

```
string toBase(ll n, int b) { string r; // n >= 0
  do { int d = n % b; r += (char)(d < 10 ? '0' + d : 'A' + d - 10); n /= b; }
  while (n);
  reverse(r.begin(), r.end()); return r; }
ll y = stoll("FF", nullptr, 16); // 255 | cout << hex << n; (dec, oct, hex stay active)
printf("%llx %llo\n", n, n); // hex and octal output of an unsigned-like value
```

All i-digit binary strings, and Gray code:

```
for (ll m = 0; m < (1LL << i); m++) cout << toBin(m, i) << '\n';
ll gray = m ^ (m >> 1); // consecutive Gray codes differ in exactly one bit
```

23. Number theory extras:

Smallest prime factor sieve (fast factorization, N up to 1e7):

```
int spf[N];
void sieveSpf() { for (int i = 2; i < N; i++) if (!spf[i])
  for (int j = i; j < N; j += i) if (!spf[j]) spf[j] = i; }
while (n > 1) { cout << spf[n] << ' '; n /= spf[n]; } // prime factors of n < N
```

Linear sieve (prime list in O(N)):

```
vector<int> pr; bool comp[N];
for (int i = 2; i < N; i++) { if (!comp[i]) pr.push_back(i);
  for (int p : pr) { if (1LL * i * p >= N) break;
    comp[i * p] = 1; if (i % p == 0) break; }}
```

Euler phi (count of 1..n coprime with n):

```
ll phi(ll n) { ll r = n;
  for (ll p = 2; p * p <= n; p++) if (n % p == 0) {
    while (n % p == 0) n /= p; r -= r / p; }
  if (n > 1) r -= r / n; return r; }
```

Extended GCD (a*x + b*y = g) and modular inverse for any modulus (gcd(a, m) = 1):

```
ll exgcd(ll a, ll b, ll &x, ll &y) {
  if (!b) { x = 1; y = 0; return a; }
  ll g = exgcd(b, a % b, y, x); y -= a / b * x; return g; }
ll inv(ll a, ll m) { ll x, y; exgcd(a, m, x, y); return (x % m + m) % m; }
```

Factorials + nCr mod p (build once, O(1) per query; raise N if n > 1e5):

```
ll f[N], fi[N];
void initFact() { f[0] = 1; for (int i = 1; i < N; i++) f[i] = f[i - 1] * i % MOD;
  fi[N - 1] = modpow(f[N - 1], MOD - 2, MOD);
  for (int i = N - 1; i > 0; i--) fi[i - 1] = fi[i] * i % MOD; }
ll C(int n, int r) {
  return r < 0 || r > n ? 0 : f[n] * fi[r] % MOD * fi[n - r] % MOD; }
```

Pascal triangle (small n; exact values up to n = 60 fit in ll):
```
ll c[61][61]; for (int i = 0; i <= 60; i++) { c[i][0] = 1;
  for (int j = 1; j <= i; j++) c[i][j] = c[i - 1][j - 1] + c[i - 1][j]; }
```

Huge exponent a^b mod p (p prime, a % p != 0): reduce b first (Fermat):
```
ll e = b % (MOD - 1); ll ans = modpow(a, e, MOD); // a % p == 0 -> answer is 0 (b > 0)
```

Miller-Rabin (deterministic for every n < 2^64):
```
ull mulmod(ull a, ull b, ull m) { return (__uint128_t)a * b % m; }
ull pw(ull a, ull b, ull m) { ull r = 1; a %= m;
  while (b) { if (b & 1) r = mulmod(r, a, m); a = mulmod(a, a, m); b >>= 1; }
  return r; }
bool isPrime(ull n) { if (n < 2) return false;
  for (ull p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
    if (n % p == 0) return n == p;
  ull d = n - 1; int s = 0; while (d % 2 == 0) d /= 2, s++;
  for (ull a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
    ull x = pw(a, d, n); if (x == 1 || x == n - 1) continue; bool ok = false;
    for (int r = 1; r < s && !ok; r++) { x = mulmod(x, x, n); ok = x == n - 1; }
    if (!ok) return false; }
  return true; }
```

Matrix exponentiation (Fibonacci: [[1,1],[1,0]]^n gives F(n) at [0][1]):
```
typedef vector<vector<ll>> Mat;
Mat mul(const Mat &A, const Mat &B) { int n = A.size(); Mat C(n, vector<ll>(n));
  for (int i = 0; i < n; i++) for (int k = 0; k < n; k++) for (int j = 0; j < n; j++)
    C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
  return C; }
Mat mpow(Mat A, ll p) { int n = A.size(); Mat R(n, vector<ll>(n));
  for (int i = 0; i < n; i++) R[i][i] = 1;
  while (p) { if (p & 1) R = mul(R, A); A = mul(A, A); p >>= 1; }
  return R; }
```

Divisor counts for every number up to N (harmonic loop, O(N log N)):
```
for (int i = 1; i < N; i++) for (int j = i; j < N; j += i) cnt[j]++; // cnt = #divisors
```

24. Graph algorithms extras:

Kruskal MST (uses the DSU above; E holds {w, u, v}):
```
vector<array<ll, 3>> E; sort(E.begin(), E.end()); ll mst = 0; dsu_init(n);
for (auto [w, u, v] : E) if (find_set(u) != find_set(v)) { union_set(u, v); mst += w; }
```

Prim MST (g[u] holds {v, w}); start from node 1, graph must be connected:
```
vector<pll> g[N]; vector<bool> used(n + 1); ll tot = 0;
priority_queue<pll, vector<pll>, greater<pll>> pq; pq.push({0, 1});
while (!pq.empty()) { auto [w, u] = pq.top(); pq.pop(); if (used[u]) continue;
  used[u] = 1; tot += w; for (auto [v, c] : g[u]) if (!used[v]) pq.push({c, v}); }
```

0-1 BFS (edge weights only 0 or 1, adj[u] holds {v, w}), O(V + E):
```
deque<int> dq; vector<int> d(n + 1, INF); d[s] = 0; dq.push_front(s);
while (!dq.empty()) { int u = dq.front(); dq.pop_front();
  for (auto [v, w] : adj[u]) if (d[u] + w < d[v]) {
    d[v] = d[u] + w; if (w) dq.push_back(v); else dq.push_front(v); }}
```

Dijkstra path restore (add par[] to the Dijkstra above, par[src] = -1):
```
par[v] = u; // inside the relaxation, right after dist[v] = dist[u] + w
vector<int> path; for (int x = target; x != -1; x = par[x]) path.push_back(x);
reverse(path.begin(), path.end()); // dist[target] == INF means unreachable
```

Bipartite check (BFS 2-coloring, color[] = -1 at start):
```
bool bip(int s) { queue<int> q; q.push(s); color[s] = 0;
  while (!q.empty()) { int u = q.front(); q.pop();
    for (int v : adj[u]) { if (color[v] == -1) { color[v] = color[u] ^ 1; q.push(v); }
      else if (color[v] == color[u]) return false; }}
  return true; }
```

Tree: subtree sizes, depth and diameter (two BFS: farthest node from any node, then farthest from that node = diameter length):

```
int sub[N], dep[N]; // call dfs(root, 0), dep[root] = 0
void dfs(int u, int p) { sub[u] = 1;
  for (int v : adj[u]) if (v != p) {
    dep[v] = dep[u] + 1; dfs(v, u); sub[u] += sub[v]; }}
```

LCA with binary lifting (O(log n) per query, call dfsL(1, 1) first):

```
int up[N][LOG], dep[N]; // dist(a, b) = dep[a] + dep[b] - 2 * dep[lca(a, b)]
void dfsL(int u, int p) { up[u][0] = p;
  for (int j = 1; j < LOG; j++) up[u][j] = up[up[u][j - 1]][j - 1];
  for (int v : adj[u]) if (v != p) { dep[v] = dep[u] + 1; dfsL(v, u); }}
int lca(int a, int b) { if (dep[a] < dep[b]) swap(a, b);
  for (int j = 0; j < LOG; j++) if ((dep[a] - dep[b]) >> j & 1) a = up[a][j];
  if (a == b) return a;
  for (int j = LOG - 1; j >= 0; j--)
    if (up[a][j] != up[b][j]) a = up[a][j], b = up[b][j];
  return up[a][0]; }
```

Strongly connected components (Kosaraju; g = graph, gr = reversed graph):

```
vector<int> g[N], gr[N], ord; bool vis[N]; int comp[N];
void d1(int u) { vis[u] = 1; for (int v : g[u]) if (!vis[v]) d1(v); ord.push_back(u); }
void d2(int u, int c) { comp[u] = c; for (int v : gr[u]) if (!comp[v]) d2(v, c); }
for (int i = 1; i <= n; i++) if (!vis[i]) d1(i);
reverse(ord.begin(), ord.end()); int c = 0; for (int u : ord) if (!comp[u]) d2(u, ++c);
```

Bridges (Tarjan, simple graph without parallel edges; tin[] = 0 at start):

```
int tin[N], low[N], timer_;
void br(int u, int p) { tin[u] = low[u] = ++timer_;
  for (int v : adj[u]) { if (v == p) continue;
    if (tin[v]) low[u] = min(low[u], tin[v]);
    else { br(v, u); low[u] = min(low[u], low[v]);
      if (low[v] > tin[u]) { /* edge u-v is a bridge */ }}}}
```

Articulation point: same low[] idea, u is one when low[v] >= tin[u] (root: needs 2+ children).

25. Dynamic programming classics:

0/1 knapsack (1D, go backwards so each item is used once):

```
vector<ll> dp(W + 1, 0);
for (int i = 0; i < n; i++) for (int w = W; w >= wt[i]; w--)
  dp[w] = max(dp[w], dp[w - wt[i]] + val[i]); // answer dp[W]
```

Unbounded knapsack / coin change (go forwards):

```
dp[0] = 0; for (int s = 1; s <= S; s++) dp[s] = INF; // minimum coins
for (int c : coins) for (int s = c; s <= S; s++) dp[s] = min(dp[s], dp[s - c] + 1);
dp[0] = 1; // number of ways (combinations): coins in the OUTER loop
for (int c : coins) for (int s = c; s <= S; s++) dp[s] = (dp[s] + dp[s - c]) % MOD;
```

For ordered ways (permutations) put the sum loop outside and the coin loop inside.

LIS in O(n log n) (strictly increasing; use upper_bound for non-decreasing):

```
vector<int> t;
for (int x : a) { auto it = lower_bound(t.begin(), t.end(), x);
  if (it == t.end()) t.push_back(x); else *it = x; } // length = t.size()
```

LCS and edit distance (dp[i][0] / dp[0][j] = 0 for LCS, = i / = j for edit distance):

```
for (int i = 1; i <= n; i++) for (int j = 1; j <= m; j++)
  dp[i][j] = a[i - 1] == b[j - 1] ? dp[i - 1][j - 1] + 1 : max(dp[i - 1][j], dp[i][j -
      1]);
for (int i = 1; i <= n; i++) for (int j = 1; j <= m; j++) // edit distance
  dp[i][j] = a[i - 1] == b[j - 1] ? dp[i - 1][j - 1]
    : 1 + min({dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]});
```

Grid paths (right / down moves, '#' = blocked), count mod MOD:

```
dp[0][0] = 1;
for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) { if (g[i][j] == '#') {
    dp[i][j] = 0; continue; }
  if (i) dp[i][j] += dp[i - 1][j]; if (j) dp[i][j] += dp[i][j - 1]; dp[i][j] %= MOD; }
```

Bitmask DP - travelling salesman (n <= 20), dp[mask][last] = min cost, start at 0:

```
for (auto &r : dp) fill(r.begin(), r.end(), INF); dp[1][0] = 0;
for (int mask = 1; mask < (1 << n); mask++) for (int i = 0; i < n; i++)
  if ((mask >> i & 1) && dp[mask][i] < INF) for (int j = 0; j < n; j++)
    if (!(mask >> j & 1)) dp[mask | 1 << j][j] = min(dp[mask | 1 << j][j], dp[mask][i]
        + w[i][j]);
```

Answer = min over i of dp[(1 << n) - 1][i] + w[i][0]. Memory: 2^n * n ints.

Subset sum with bitset (reachable sums, fast for sums up to ~1e6):

```
bitset<1000001> b; b[0] = 1; for (int x : a) b |= b << x; // b[s] = 1 if sum s is
    // possible
```

Memoization template and digit DP (count numbers <= N with digit 1, N as string s):

```
memset(memo, -1, sizeof memo); // ll memo[20][2][20]
ll go(int pos, bool tight, int cnt) { if (pos == (int)s.size()) return cnt;
  ll &r = memo[pos][tight][cnt]; if (r != -1) return r; ll res = 0;
  for (int d = 0, lim = tight ? s[pos] - '0' : 9; d <= lim; d++)
    res += go(pos + 1, tight && d == lim, cnt + (d == 1));
  return r = res; }
```

DP over intervals / partitions (shape to remember):

```
for (int len = 2; len <= n; len++) for (int l = 0; l + len - 1 < n; l++) { int r = l +
    len - 1;
  for (int k = l; k < r; k++) dp[l][r] = min(dp[l][r], dp[l][k] + dp[k + 1][r] +
      cost(l, r)); }
```

26. Advanced data structures:

Fenwick tricks: range add + point query -> add(l, v); add(r + 1, -v); value = sum(i).
k-th smallest with BIT descent (bit[] holds counts, LOGN = highest power of 2 <= n):

```
int kth(int k) { int pos = 0;
  for (int pw = LOGN; pw; pw >>= 1) if (pos + pw <= n && bit[pos + pw] < k) {
    pos += pw; k -= bit[pos]; }
  return pos + 1; }
```

Segment tree with lazy propagation (range add, range sum; node 1 covers [0, n-1]):

```
ll t[4 * N], lz[4 * N];
void push(int x, int l, int r) { if (!lz[x]) return; int m = (l + r) / 2;
  t[2 * x] += lz[x] * (m - l + 1); lz[2 * x] += lz[x];
  t[2 * x + 1] += lz[x] * (r - m); lz[2 * x + 1] += lz[x]; lz[x] = 0; }
void upd(int x, int l, int r, int ql, int qr, ll v) { if (qr < l || r < ql) return;
  if (ql <= l && r <= qr) { t[x] += v * (r - l + 1); lz[x] += v; return; }
  push(x, l, r); int m = (l + r) / 2; upd(2 * x, l, m, ql, qr, v);
  upd(2 * x + 1, m + 1, r, ql, qr, v); t[x] = t[2 * x] + t[2 * x + 1]; }
ll qry(int x, int l, int r, int ql, int qr) { if (qr < l || r < ql) return 0;
  if (ql <= l && r <= qr) return t[x]; push(x, l, r); int m = (l + r) / 2;
  return qry(2 * x, l, m, ql, qr) + qry(2 * x + 1, m + 1, r, ql, qr); }
```

Starts with all zeros; for an initial array call upd(1, 0, n - 1, i, i, a[i]) for each i.

Binary trie for maximum XOR (31 bits; for words use [26] children and letters):

```
int tr[N * 31][2], cnt = 1; // node 0 = root
void ins(int x) { int u = 0; for (int b = 30; b >= 0; b--) { int c = x >> b & 1;
  if (!tr[u][c]) tr[u][c] = cnt++; u = tr[u][c]; }}
int maxXor(int x) { int u = 0, r = 0; for (int b = 30; b >= 0; b--) { int c = (x >> b
    & 1) ^ 1;
  if (tr[u][c]) { r |= 1 << b; u = tr[u][c]; } else u = tr[u][c ^ 1]; }
  return r; }
```

Ordered set (k-th element and rank in O(log n)) - GNU policy based data structure:

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef tree<ll, null_type, less<ll>, rb_tree_tag, tree_order_statistics_node_update>
    oset;
oset s; s.insert(5); s.order_of_key(x); // how many elements are < x
*s.find_by_order(k); // k-th smallest, 0-indexed (returns s.end() if k >= size)
```

Duplicates: store pair<ll, int>(value, unique id). Do not use less_equal (erase breaks).

Random numbers and shuffle (fixed seed is hackable, so use the clock):

```
mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ll r = rng() % n; // 0..n-1 | uniform_int_distribution<int>(l, r)(rng) // l..r
shuffle(v.begin(), v.end(), rng); // shuffle before sort() if the tests may be
    // anti-quicksort
```

Mo's algorithm idea (offline range queries, O((n + q) sqrt n)):

Sort queries by (l / B, r) with B = sqrt(n); keep a window [L, R]; move L and R one step at a time with add(i) / remove(i); the answer of a query is the current state of the window.

27. String algorithms extras:

Z-function (z[i] = longest common prefix of s and s[i..]); pattern search: z of pat + '#' + text:

```
vector<int> zf(const string &s) { int n = s.size(); vector<int> z(n);
  for (int i = 1, l = 0, r = 0; i < n; i++) { if (i < r) z[i] = min(r - i, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] > r) l = i, r = i + z[i]; }
  return z; }
```

A match starts at every position of the text where z == pattern length. Period of s: smallest p with z[p] == n - p.

Manacher (all palindromic substrings; d1 = odd centres, d2 = even centres):

```
vector<int> d1(n), d2(n);
for (int i = 0, l = 0, r = -1; i < n; i++) { int k = i > r ? 1 : min(d1[l + r - i], r
    - i + 1);
  while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
  d1[i] = k--; if (i + k > r) l = i - k, r = i + k; }
for (int i = 0, l = 0, r = -1; i < n; i++) { int k = i > r ? 0 : min(d2[l + r - i +
    1], r - i + 1);
  while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
  d2[i] = k--; if (i + k > r) l = i - k - 1, r = i + k; }
```

Number of palindromic substrings = sum of d1[i] + sum of d2[i].

Split a line into words / by a separator:

```
stringstream ss(line); string w; while (ss >> w) { /* word */ }
while (getline(ss, w, ',')) { /* piece between commas */ }
```

Small string tricks:

```
bool pal = (s == string(s.rbegin(), s.rend())); // palindrome check
int k = s[i] - 'a'; char c2 = 'a' + k; // letter <-> index | caesar shift: 'a' + (s[i]
    // - 'a' + x) % 26
bool sub = false; for (int i = 0, j = 0; i < (int)t.size(); i++) if (j < (int)s.size()
    && t[i] == s[j]) j++;
```

After the loop: s is a subsequence of t exactly when j == s.size().

```
string rep = string(5, 'x'); s.insert(pos, "ab"); s.erase(pos, len); s.replace(pos,
    len, "z");
s.find('c', from); s.rfind("ab"); s.find_first_of("aeiou"); s.compare(t); s += t;
    s.back(); s.front();
```

28. Geometry and miscellaneous math:

Basics (use ll while all coordinates are integers; double only at the end):

```
double dist = hypot(x1 - x2, y1 - y2); double ang = atan2(y, x); const double PI =
    acos(-1.0);
ll cross(ll ax, ll ay, ll bx, ll by) { return ax * by - ay * bx; } // > 0 means b is
    // left of a
ll area2 = 0; for (int i = 0; i < n; i++) { int j = (i + 1) % n; area2 += x[i] * y[j]
    - x[j] * y[i]; }
```

Polygon area = abs(area2) / 2.0 (shoelace). Pick: A = I + B / 2 - 1 (I inside, B on boundary lattice points).

Calendar and rounding helpers:

```
bool leap = (y % 4 == 0 && y % 100 != 0) || y % 400 == 0; // Feb has 29 days
int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}; // 1900 and 2100 are
    // NOT leap
ll fdiv(ll a, ll b) { return a / b - ((a % b != 0) && ((a < 0) != (b < 0))); } // floor
ll cdiv(ll a, ll b) { return a / b + ((a % b != 0) && ((a < 0) == (b < 0))); } // ceil
ll md(ll a, ll m) { return (a % m + m) % m; } // mod that is never negative
ll r = llround(x); ll f = floor(x); ll cl = ceil(x); // round-half-away, floor, ceil
    // on doubles
```

Convex hull (Andrew monotone chain, O(n log n), keeps strictly convex corners):

```
typedef pair<ll, ll> P;
ll cr(P o, P a, P b) { return (a.fi - o.fi) * (b.se - o.se) - (a.se - o.se) * (b.fi -
    o.fi); }
vector<P> hull(vector<P> p) { sort(p.begin(), p.end()); p.erase(unique(p.begin(),
    p.end()), p.end());
  int n = p.size(), k = 0; if (n < 3) return p; vector<P> h(2 * n);
  for (int i = 0; i < n; i++) { while (k >= 2 && cr(h[k - 2], h[k - 1], p[i]) <= 0)
      k--; h[k++] = p[i]; }
  for (int i = n - 2, t = k + 1; i >= 0; i--) {
    while (k >= t && cr(h[k - 2], h[k - 1], p[i]) <= 0) k--; h[k++] = p[i]; }
  h.resize(k - 1); return h; }
```

29. STL, C++ tricks and debugging:

Recursive lambda, structured bindings:

```
function<void(int, int)> dfs = [&](int u, int p) { for (int v : adj[u]) if (v != p)
    dfs(v, u); };
dfs(1, 0); // also: auto [a, b] = pr; for (auto &[k, v] : mp) { } // C++17
```

Min / max of many values, swap, tie:

```
int m = min({a, b, c}), M = max({a, b, c}); swap(a, b); tie(a, b) = make_pair(b, a);
```

Struct with operator< (works in set, map, sort, priority_queue):

```
struct E { int a, b; bool operator<(const E &o) const { return a != o.a ? a < o.a : b
    > o.b; }};
```

Custom comparator for set / priority_queue (priority_queue top = largest by cmp):

```
set<int, greater<int>> s; // descending set
struct cmp { bool operator()(const pll &x, const pll &y) const { return x.se < y.se;
    }};
set<pll, cmp> st; priority_queue<pll, vector<pll>, cmp> pq;
```

Sort indexes by value (keep original positions):

```
vector<int> id(n); iota(id.begin(), id.end(), 0);
sort(id.begin(), id.end(), [&](int x, int y) { return a[x] < a[y]; });
```

Set / map extras (erase while iterating, neighbours, extremes):

```
for (auto it = s.begin(); it != s.end(); ) { if (bad(*it)) it = s.erase(it); else
    ++it; }
auto nx = next(it); if (it != s.begin()) { auto pv = prev(it); } // neighbours of it
*s.rbegin(); *prev(s.end()); s.erase(s.begin()); mp.lower_bound(x); mp.upper_bound(x);
```

Output tricks:

```
for (int i = 0; i < n; i++) cout << v[i] << " \n"[i + 1 == n]; // no trailing space
cout << fixed << setprecision(9) << x; printf("%.9Lf\n", y); // double / long double
cout << (ll)1e18 << '\n'; // 1e18 is a double: cast it | cout << flush; // interactive
```

Overflow safe checks and limits:

```
ll c; if (__builtin_mul_overflow(a, b, &c)) { /* a * b does not fit in ll */ }
numeric_limits<ll>::max(); LLONG_MAX; INT_MIN; // abs(INT_MIN) overflows!
```

Fast reader for 1e6+ numbers (do not mix with cin / scanf):

```
int rd() { int c = getchar(), x = 0, s = 1; while (c != '-' && (c < '0' || c > '9')) c
    = getchar();
  if (c == '-') s = -1, c = getchar();
  while (c >= '0' && c <= '9') x = x * 10 + c - '0', c = getchar();
  return x * s; }
```

Debug helpers (delete or comment out before submitting) and timer:

```
#define dbg(x) cerr << #x << " = " << (x) << endl;
#define dbgv(v) { cerr << #v << ":"; for (auto &e : v) cerr << ' ' << e; cerr << endl;
    }
double sec = (double)clock() / CLOCKS_PER_SEC; // elapsed time, compare with the limit
```

Compile with warnings + sanitizers, and stress test against a brute force (shell):

```
g++ -std=c++17 -O2 -Wall -Wextra -Wshadow -fsanitize=address,undefined -g a.cpp -o a
for ((i = 1; i <= 1000; i++)); do ./gen $i > in; ./a < in > o1; ./brute < in > o2;
  cmp -s o1 o2 || { echo "diff on test $i"; break; }; done
```

gen.cpp: read the seed with atoi(argv[1]), call mt19937 rng(seed), print a SMALL random test.

30. Handy tables and formulas:

Powers of two and type limits:

2^10 = 1024 | 2^16 = 65536 | 2^20 = 1048576 | 2^30 = 1073741824 | 2^31 = 2147483648 | 2^32 = 4294967296
2^40 = 1.1e12 | 2^50 = 1.1e15 | 2^60 = 1.15e18 | 2^62 = 4.6e18 | 2^63 = 9.22e18 | 2^64 = 1.8e19
int: -2^31 .. 2^31 - 1 | unsigned: 0 .. 4294967295 | ll: -2^63 .. 2^63 - 1 | ull: 0 .. 1.8e19 | char: -128 .. 127

Factorials, Fibonacci, Catalan:

5! = 120 | 10! = 3628800 | 12! = 479001600 (largest in int) | 15! = 1.3e12 | 20! = 2432902008176640000 (largest in ll)
F(1..15) = 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 | F(30) = 832040 | F(46) = 1836311903 (largest in int)
F(92) = 7540113804746346429 (largest in ll) | F(n) = F(n-1) + F(n-2)
Catalan: 1 1 2 5 14 42 132 429 1430 4862 16796 58786 208012 742900 | Cn = C(2n, n) / (n + 1)
Catalan counts: balanced brackets with n pairs, binary trees with n nodes, triangulations of an (n+2)-gon.

Primes:

First 25: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
Big: 998244353 | 1000000007 | 1000000009 | largest below 1e5: 99991 | 1e6: 999983 | 1e7: 9999991 | 1e9: 999999937
How many primes below 10^3 / 10^4 / 10^5 / 10^6 / 10^7 / 10^8 / 10^9: 168 / 1229 / 9592 / 78498 / 664579 / 5761455 / 50847534

Most divisors for a number up to (number with that many divisors):

1e3: 32 (840) | 1e4: 64 (7560) | 1e5: 128 (83160) | 1e6: 240 (720720) | 1e9: 1344 (735134400)
1e12: 6720 (963761198400) | 1e18: 103680 (897612484786617600)

Sums and counting:

Sum 1..n = n(n+1)/2 | squares = n(n+1)(2n+1)/6 | cubes = (n(n+1)/2)^2 | odd numbers 1..2n-1 = n^2
Geometric: a + ar + ... + ar^(n-1) = a(r^n - 1)/(r - 1) | arithmetic: n-th term = a + (n-1)d
Harmonic: 1 + 1/2 + ... + 1/n ~ ln n + 0.577 | sum over i of n/i ~ n ln n (the cost of a sieve)
Stars and bars: n identical items into k boxes = C(n + k - 1, k - 1) (boxes may be empty)
Arrangements with repeats: n! / (a! b! c!) | circular: (n - 1)! | derangements D(n) = (n-1)(D(n-1) + D(n-2))
D(1..6) = 0 1 2 9 44 265 | Inclusion-exclusion: |A u B| = |A| + |B| - |A n B| | Pigeonhole: n+1 items, n boxes
Binomial: C(n,k) = C(n-1,k-1) + C(n-1,k) | sum of C(n,k) over k = 2^n | C(n,k) = C(n,n-k)

Number theory facts:

Fermat: a^(p-1) = 1 (mod p), p prime | Euler: a^phi(m) = 1 (mod m), gcd(a, m) = 1 | Wilson: (p-1)! = -1 (mod p)
gcd(a, b) = gcd(a, b - a) = gcd(b, a % b) | gcd(a, 0) = a | gcd(a, b) * lcm(a, b) = a * b
Legendre: exponent of prime p in n! = n/p + n/p^2 + ... | trailing zeros of n! = n/5 + n/25 + n/125 + ...
Digit sum mod 9 = number mod 9 | divisible by 3 / 9 <=> digit sum is | by 11: alternating digit sum
Number of divisors = product of (e + 1) over prime powers p^e | sum of divisors = product of (p^(e+1) - 1)/(p - 1)

Graph and tree facts:

Sum of degrees = 2E | tree: n - 1 edges, exactly one path between two nodes | complete graph: n(n-1)/2 edges
Labelled trees on n nodes = n^(n-2) (Cayley) | Euler (planar): V - E + F = 2 | bipartite <=> no odd cycle
A DAG has a topological order | undirected graph has an Euler circuit <=> connected and all degrees even
Edges needed to connect c components = c - 1 | a tree's diameter endpoints are leaves

Bit facts:

a + b = (a ^ b) + 2 * (a & b) | a | b = a + b - (a & b) | a ^ a = 0 | a ^ 0 = a | x & (x - 1) clears the lowest set bit
XOR of 1..n: n % 4 == 0 -> n, 1 -> 1, 2 -> n + 1, 3 -> 0 | power of two: x > 0 && (x & (x - 1)) == 0
x << k = x * 2^k | x >> k = floor(x / 2^k) | ~x = -x - 1 | -x = ~x + 1 (two's complement)
Subsets containing one fixed element: 2^(n-1) | subsets of a mask m: for (s = m; ; s = (s - 1) & m) { if (!s) break; }

ASCII:

'0' = 48, '9' = 57, 'A' = 65, 'Z' = 90, 'a' = 97, 'z' = 122, ' ' = 32, newline = 10; 'a' - 'A' = 32.

31. Contest-day checklist:

Before coding:

1. Read the statement twice; mark the constraints, the output format and words like distinct, strictly, at most.
2. Solve the sample by hand, then try tiny cases (n = 1, n = 2), the largest case and an all-equal case.
3. Pick the complexity from the constraints (section 20) and the data types (ll if a sum or product can pass 2e9).
4. Think of a slow brute force first: it checks the idea and is the base for a stress test later.

Before submitting:
Multi-test: reset every global array / vector / counter for each test (clear only the first n cells, not everything).
Output format: spaces vs newlines, YES / NO case, trailing spaces, precision of doubles (setprecision).
Indexing: 0-based vs 1-based, i < n vs i <= n, off-by-one in prefix sums, binary search and sliding windows.
Overflow: 1LL * a * b, big sums, INF + INF, factorial and power tables, (a - b) % MOD going negative.
Unsigned trap: v.size() - 1 on an empty vector is huge; compare with (int)v.size() instead.
Uninitialised variables; big arrays must be global (the stack is only about 8 MB).
Recursion depth: DFS on a 2e5-node chain can crash; use an iterative DFS / BFS or reduce the depth.
Hacks: unordered_map needs the chash, rand() with a fixed seed is predictable (seed with the clock).
Comparator must be a strict weak order: never use <= and never return true for equal elements.
Floating point: avoid == on doubles, use eps = 1e-9; for exact answers use integer math (sqrt can be off by one).
Reading: ll for values up to 1e18; strings with cin >> s; after cin >> n use cin.ignore() before getline.

If you are stuck:
Wrong Answer -> stress test against a brute force on small random inputs (section 29), print intermediate values.
Time Limit -> count operations (n * q?), remove a log with two pointers / prefix sums / DSU, avoid endl and map.
No idea -> shrink n, look for monotonicity (binary search on the answer), sorting, greedy exchange, work backwards.
Reduce the problem: fix one value and count the rest, think per bit, per element contribution, or per prefix.

32. Search, greedy and counting patterns:

Ternary search (maximum of a unimodal function) and binary search on doubles:
```
double lo = 0, hi = 1e9;
for (int it = 0; it < 100; it++) { double m1 = lo + (hi - lo) / 3, m2 = hi - (hi - lo)
    / 3;
  if (f(m1) < f(m2)) lo = m1; else hi = m2; } // maximum of f is near lo
for (int it = 0; it < 100; it++) { double mid = (lo + hi) / 2; if (ok(mid)) hi = mid;
    else lo = mid; }
```

Activity selection (most non-overlapping intervals: sort by END time):
```
sort(v.begin(), v.end(), [](pll a, pll b) { return a.se < b.se; }); ll last = -LINF,
    cnt = 0;
for (auto [s, e] : v) if (s >= last) { cnt++; last = e; } // s > last if touching ones
    // overlap
```

Maximum overlap of closed intervals (sweep line over events):
```
vector<pair<int, int>> ev; for (auto [l, r] : segs) { ev.push_back({l, 1});
    ev.push_back({r + 1, -1}); }
sort(ev.begin(), ev.end()); int cur = 0, best = 0;
for (auto [x, d] : ev) { cur += d; best = max(best, cur); }
```

Cheapest way to merge items (Huffman style: always take the two smallest):
```
priority_queue<ll, vector<ll>, greater<ll>> pq; for (ll x : a) pq.push(x); ll cost = 0;
while (pq.size() > 1) { ll x = pq.top(); pq.pop(); ll y = pq.top(); pq.pop(); cost +=
    x + y; pq.push(x + y); }
```

Subarrays with sum = k (prefix sums + map, works with negatives):
```
map<ll, ll> seen; seen[0] = 1; ll pre = 0, ans = 0;
for (ll x : a) { pre += x; ans += seen[pre - k]; seen[pre]++; }
```

Subarrays with at most k distinct values (exactly k = atMost(k) - atMost(k - 1)):
```
map<int, int> fr; ll ans = 0; int l = 0;
for (int r = 0; r < n; r++) { fr[a[r]]++;
  while ((int)fr.size() > k) { if (--fr[a[l]] == 0) fr.erase(a[l]); l++; }
  ans += r - l + 1; }
```

Longest substring without a repeated character:
```
int last[256]; fill(last, last + 256, -1); int best = 0, l = 0;
for (int r = 0; r < n; r++) { l = max(l, last[(unsigned char)s[r]] + 1);
  last[(unsigned char)s[r]] = r; best = max(best, r - l + 1); }
```

Next greater element index (nge[i] = n when none) and largest rectangle in a histogram:

```
vector<int> nge(n, n); stack<int> st;
for (int i = 0; i < n; i++) { while (!st.empty() && a[st.top()] < a[i]) {
    nge[st.top()] = i; st.pop(); }
  st.push(i); }
a.push_back(0); stack<int> s2; ll best = 0; // histogram: a[i] = bar heights
for (int i = 0; i < (int)a.size(); i++) { while (!s2.empty() && a[s2.top()] >= a[i]) {
    ll h = a[s2.top()]; s2.pop(); ll l = s2.empty() ? -1 : s2.top(); best = max(best,
        h * (i - l - 1)); }
  s2.push(i); }
```

Count inversions with the BIT (a[i] compressed to 1..m, BIT size n = m):

```
ll inv = 0; for (int i = n - 1; i >= 0; i--) { inv += sum(a[i] - 1); add(a[i], 1); }
```

Euler tour (subtree = one contiguous range of the flat array):

```
int tin[N], tout[N], flat[N], tim = 0;
void euler(int u, int p) { tin[u] = ++tim; flat[tim] = u;
  for (int v : adj[u]) if (v != p) euler(v, u); tout[u] = tim; }
```

Subtree of u = positions tin[u]..tout[u] -> range query with BIT / segment tree.
u is an ancestor of v <=> tin[u] <= tin[v] && tout[v] <= tout[u].

Cycle detection in a directed graph (colors) and multi-source BFS:

```
int col[N]; // 0 = new, 1 = on the current path, 2 = finished
bool cyc(int u) { col[u] = 1; for (int v : g[u]) { if (col[v] == 1) return true;
    if (!col[v] && cyc(v)) return true; }
  col[u] = 2; return false; }
```

Multi-source BFS: push ALL sources with dist = 0 into the queue first, then run the normal BFS.

33. Backtracking, game theory and counting tricks:

All subsets / all permutations by recursion (cur = current choice, used[] for permutations):

```
void sub(int i) { if (i == n) { /* use cur */ return; }
  cur.push_back(a[i]); sub(i + 1); cur.pop_back(); sub(i + 1); }
void perm() { if ((int)cur.size() == n) { /* use cur */ return; }
  for (int i = 0; i < n; i++) if (!used[i]) {
    used[i] = 1; cur.push_back(a[i]); perm(); cur.pop_back(); used[i] = 0; }}
```

N-Queens (count the solutions; rows tried one by one):

```
int cntq = 0; bool col[20], d1[40], d2[40];
void q(int r) { if (r == n) { cntq++; return; }
  for (int c = 0; c < n; c++) if (!col[c] && !d1[r + c] && !d2[r - c + n]) {
    col[c] = d1[r + c] = d2[r - c + n] = 1; q(r + 1);
    col[c] = d1[r + c] = d2[r - c + n] = 0; }}
```

Game theory (two players, optimal play, last mover wins):

Nim: the first player wins <=> XOR of all pile sizes != 0. One pile where you may take 1..k: losing when n % (k + 1) == 0.
Sprague-Grundy: g(state) = mex of g(next states); a sum of independent games is a Nim sum (XOR of their g values).

```
int mex = 0; while (st.count(mex)) mex++; // st = set of Grundy values of the next
    // states
```

If every move is allowed from a small state, brute force the win / lose table with DP and look for a pattern.

Josephus (0-indexed survivor of n people, every k-th removed):

```
int jos(int n, int k) { int r = 0; for (int i = 2; i <= n; i++) r = (r + k) % i;
    return r; }
```

Meet in the middle (n <= 40 items):

Split into two halves of 20; list all 2^20 subset sums of each half; sort one list; for every sum x of the other list binary search the partner (target - x). Total O(2^(n/2) * n) instead of O(2^n).

Counting and expectation habits:

Count the answer as a sum of contributions: for each element / pair / bit count how many times it is used.
Count the complement when the opposite is easier (all - bad). Fix one parameter, count the rest with a formula.
Linearity of expectation: E[X + Y] = E[X] + E[Y] always. Use indicator variables: E[count] = sum of P(event i).
Expected number of tries until a success with probability p = 1 / p. Probabilities mod p: use modular inverses.
Per bit trick: for XOR / AND / OR sums over many numbers solve each of the 30 bits independently.
Sort first when order does not matter: it exposes two pointers, binary search, greedy and neighbours.

Sqrt decomposition (block size B = sqrt(n)), when a segment tree is too heavy to write:

```
int B = 320; // point update: a[i] = v; blk[i / B] += v - old; range sum over [l, r]:
ll rs(int l, int r) { ll s = 0; while (l <= r && l % B) s += a[l++];
  while (l + B - 1 <= r) { s += blk[l / B]; l += B; }
  while (l <= r) s += a[l++];
  return s; }
```

34. More STL algorithms and small helpers:

STL algorithms (v = vector; <algorithm> and <numeric>):

```
all_of(v.begin(), v.end(), [](int x) { return x > 0; }); // any_of / none_of work the
    // same way
count_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; }); // find_if returns an
    // iterator
v.erase(remove_if(v.begin(), v.end(), [](int x) { return x < 0; }), v.end()); //
    // delete by condition
is_sorted(v.begin(), v.end()); minmax_element(v.begin(), v.end()); // {min it, max it}
auto er = equal_range(v.begin(), v.end(), x); // {lower_bound, upper_bound}, count =
    // er.second - er.first
vector<int> r; set_intersection(a.begin(), a.end(), b.begin(), b.end(),
    back_inserter(r));
merge(a.begin(), a.end(), b.begin(), b.end(), back_inserter(r)); // set_union /
    // set_difference alike
adjacent_difference(a.begin(), a.end(), d.begin()); // d[i] = a[i] - a[i - 1];
    // partial_sum reverses it
transform(a.begin(), a.end(), a.begin(), [](int x) { return x * 2; });
    replace(a.begin(), a.end(), 1, 9);
```

Sum of floor(n / i) for i = 1..n in O(sqrt n) (blocks with the same quotient):

```
ll ans = 0; for (ll l = 1, r; l <= n; l = r + 1) { r = n / (n / l); ans += (r - l + 1)
    * (n / l); }
```

Numbers in [1, n] divisible by a or by b (inclusion-exclusion):

```
ll cntab = n / a + n / b - n / lcm(a, b);
```

Matrix transpose and rotate 90 degrees clockwise (a is n x m, result is m x n):

```
vector<vector<int>> t(m, vector<int>(n)), r(m, vector<int>(n));
for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) { t[j][i] = a[i][j]; r[j][n -
    1 - i] = a[i][j]; }
```

Knight moves, cell id <-> (i, j):

```
int kx[] = {1, 2, 2, 1, -1, -2, -2, -1}, ky[] = {2, 1, -1, -2, -2, -1, 1, 2};
int id = i * m + j; // cell -> number | back: i = id / m, j = id % m
```

Hash for pair keys in unordered_set / unordered_map (uses chash from section 9):

```
struct PH { size_t operator()(const pair<int, int> &p) const {
  return chash()((ull)p.first << 32 ^ (unsigned)p.second); }};
unordered_set<pair<int, int>, PH> us;
```

Heap with deletion: multiset, or lazy deletion with a second heap:

```
multiset<ll> ms; ms.insert(x); ms.erase(ms.find(x)); *ms.begin(); *ms.rbegin(); // min
    // / max
```

Lazy delete: keep priority_queue<ll> pq, del; erase x with del.push(x); before reading pq.top() run
while (!del.empty() && pq.top() == del.top()) { pq.pop(); del.pop(); }

Small number tricks (floor log2, next power of two, exact integer square root):

```
int fl = 31 - __builtin_clz(x); // floor(log2 x), x > 0
int up = 1 << (32 - __builtin_clz(x - 1)); // smallest power of two >= x (x > 1)
ll sq = sqrtl(n); while (sq * sq > n) sq--; while ((sq + 1) * (sq + 1) <= n) sq++; //
    // isqrt
```

DP memory: rolling array when dp[i] needs only dp[i - 1]:

Keep two rows and swap them (swap(cur, prev)), or index with [i & 1]; 1D knapsack is already a rolling array.